Third Year Engineering

18BTCS601-Design and Analysis of Algorithms

Class - T.Y. (SEM-I)

Unit - 2

Brute Force and Divide-and-Conquer

AY 2023-2024 SEM-I

Unit-II Syllabus

Brute Force: Principle, Brute force string matching algorithm and its analysis, NAÏVE string matching algorithms and its analysis, Rabin Karp algorithm and its analysis; **Divide-and-Conquer Methodology:** Principle, algorithms: Binary Search and its analysis, Merge sort and its analysis, Quick sort and its analysis, Stassen's matrix multiplication

Brute Force: Principle

- A Brute Force Algorithm is the straightforward approach to a problem i.e., the first approach that comes to our mind on seeing the problem.
- More technically it is just like iterating every possibility available to solve that problem.

- Ex: Searching a String Pattern in Given Text
- Calculating Power of N
- Sorting an elements etc.

when the problem size is limited it is useful.

The method is also used when the simplicity of implementation is more important than speed.

Brute force string matching algorithm and its analysis

What is String Matching:

Identify the index of String (m) in Given Text (P)

where m < p

Search for string ‘ NOT ‘ in the Text “ NOBODY NOTICED HIM’

```
N O B O D Y _ N O T I C E D _ H I M
N O T
  N O T
    N O T
      N O T
        N O T
          N O T
            N O T
              N O T
```

String Match

- Also known as Substring Search

n m $m \leq n$

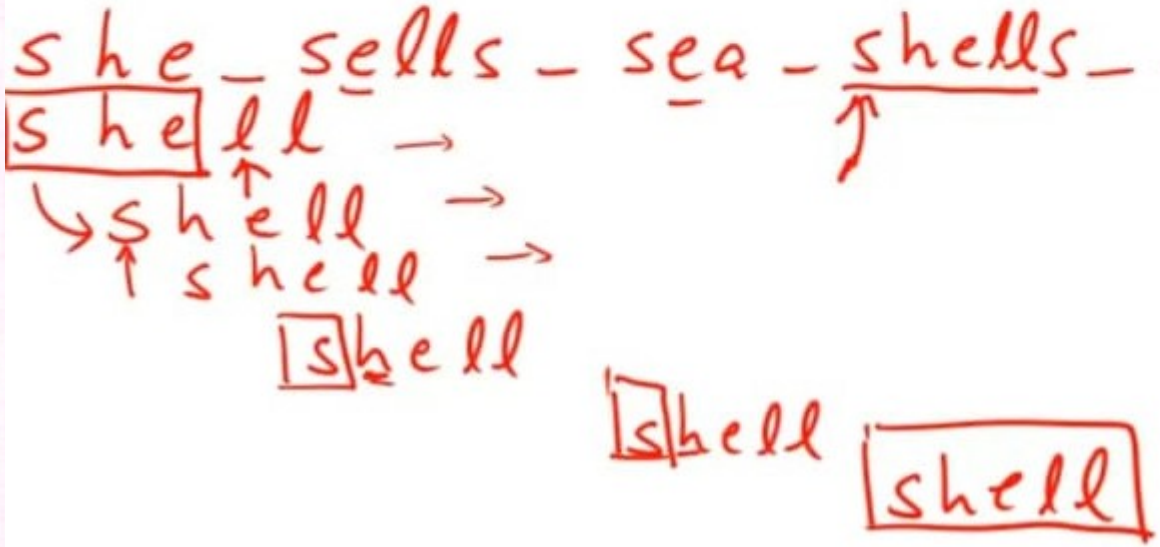
- Given a text T and a pattern P,
  - Is the pattern a substring of the text, i.e.,
  - Is there a position i where the entire pattern occurs in the given text?

- A brute-force algorithm
- Must ignore partial matches

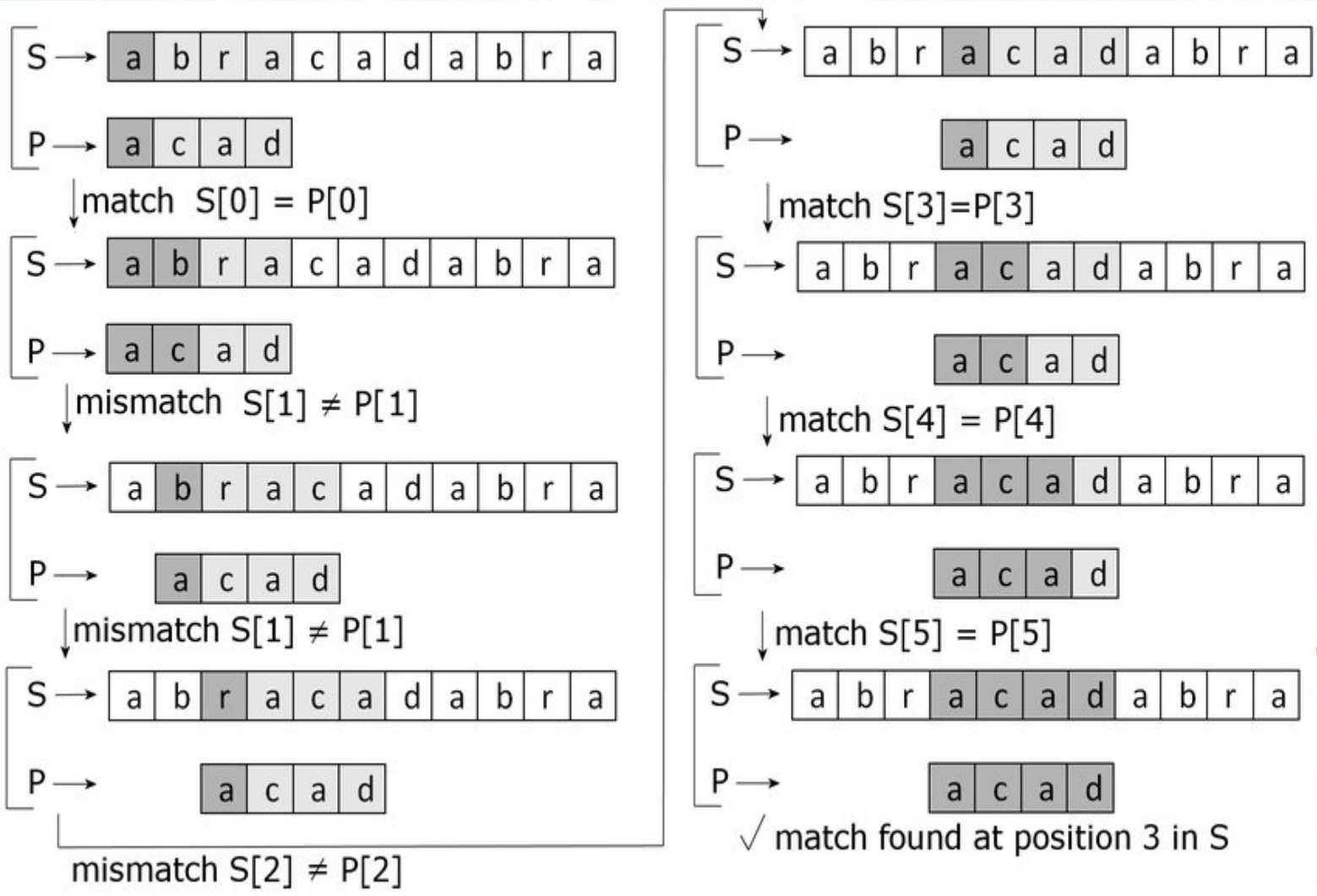
Example

- Search for "shell" in

  "she sells sea shells on the sea shore"



```
function bruteForcePatternMatch(T, P):
    n = length(T)
    m = length(P)

    for i from 0 to n - m:
        j = 0
        while j < m and T[i + j] == P[j]:
            j = j + 1
        if j == m:
            return i  // Pattern found at position i
    return -1  // Pattern not found
```


S → a b r a c a d a b r a
P → a c a d
match S[0] = P[0]
S → a b r a c a d a b r a
P → a c a d
mismatch S[1] ≠ P[1]
S → a b r a c a d a b r a
P → a c a d
mismatch S[1] ≠ P[1]
S → a b r a c a d a b r a
P → a c a d
mismatch S[2] ≠ P[2]
S → a b r a c a d a b r a
P → a c a d
match S[3]=P[3]
S → a b r a c a d a b r a
P → a c a d
match S[4] = P[4]
S → a b r a c a d a b r a
P → a c a d
match S[5] = P[5]
S → a b r a c a d a b r a
P → a c a d
√ match found at position 3 in S

Analysis: How Long Does it Take?

- How many times do we compare?

$m \times (n - m + 1)$ $\quad m \leq n$

$O(n \times m)$

How many times do we compare?

$m \times (n - m + 1)$

$m \leq n$

$O(n \times m)$

If $m << n$,

$m \leq 10$

$O(n)$

Random case:

$\Theta(n + m)$

$m = n$

$O(n^2)$

NAÏVE string matching algorithms and its analysis

- Finding all occurrences of a pattern in a given text(or body of text).
- Naive algorithm is exact string matching(means finding one or all exact occurrences of a pattern in a text) algorithm.
- The naive approach tests all the possible placement of Pattern P [1.....m] relative to text T [1.....n].
- We try shift s = 0, 1.....n-m, successively and for each shift s.
- Compare T [s+1.....s+m] to P [1.....m].It returns all the valid shifts found.

NAÏVE string matching algorithms and its analysis

Example:

Input: txt[] = "THIS IS STRING MATCHING ALGORITHM"

pat[] = "STRING"

Output: Pattern found at position 10

NAÏVE string matching algorithms and its analysis

Example :

Input:

Main String: “ABAAABCDBBABCDDEBCABC”

pattern: “ABC”

Output:

Pattern found at position: 4

Pattern found at position: 10

Pattern found at position: 18

NAÏVE string matching algorithm

```
NAIVE-STRING-MATCHER (T, P)
1. n ← length [T]
2. m ← length [P]
3. for s ← 0 to n -m
4. do if P [1.....m] = T [s + 1....s + m]
5. then print "Pattern occurs with shift"
```

Analysis: This for loop from 3 to 5 executes for n-m + 1(we need at least m characters at the end) times and in iteration we are doing m comparisons. So the total complexity is O (n-m+1).

NAÏVE string matching algorithm analysis

What is the best case?

The best case occurs when the first character of the pattern is not present in text at all.

txt[] = "BBACCAADDEE";

pat[] = "HBB";

The number of comparisons in best case is O(n).

NAÏVE string matching algorithm analysis

What is the worst case ?

The worst case of Naive Pattern Searching occurs in following scenarios.

1) When all characters of the text and pattern are same.

```
txt[] = "DDDDDDDDDDDD";
pat[] = "DDDDD";
```

2) Worst case also occurs when only the last character is different.

```
txt[] = "VVVVVVVVVVVK";
pat[] = "VVVK";
```

The number of comparisons in the worst case is O(m*(n-m+1)).

Rabin Karp algorithm and its analysis;

It is String matching algorithm.

It is another application of Hashing.

The Rabin-Karp string searching algorithm calculates a hash value for the pattern, and for each M-character subsequence of text to be compared.

If the hash values are unequal, the algorithm will calculate the hash value for next M-character sequence.

If the hash values are equal, the algorithm will compare the pattern and the M-character sequence.

In this way, there is only one comparison per text subsequence, and character matching is only needed when hash values match.

Rabin Karp algorithm

RABIN-KARP-MATCHER (T, P, d, q)

1. n ← length [T]
2. m ← length [P]
3. h ← d mod q
4. p ← 0
5. t_0 ← 0
6. for i ← 1 to m
7. do p ← (dp + P[i]) mod q
8. t_0 ← (dt_0+T [i]) mod q
9. for s ← 0 to n-m
10. do if p = t_s
11. then if P [1.....m] = T [s+1.....s + m]
12. then "Pattern occurs with shift" s
13. If s < n-m
14. then t_{s+1} ← (d (t_s-T [s+1]h)+T [s+m+1])mod q

Rabin Karp algorithm

Example-1

- Given **T = 3 1 4 1 5 9 2 6 5 3 5** and **P = 26**
- Now find hash value : **p mod q = 26 mod 11 = 4**
  - Here we have taken q=11 (prime number)

3	1	4	1	5	9	2	6	5	3	5

31 mod 11 = 9 , here **9 is not equal to 4**. **So Don't Compare**

3	1	4	1	5	9	2	6	5	3	5

14 mod 11 = 3 , here **3 is not equal to 4**. **So don't compare**

3	1	4	1	5	9	2	6	5	3	5

41 mod 11 = 8 , here **8 is not equal to 4**. **So don't compare.**

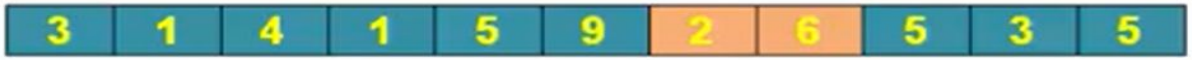
Rabin Karp algorithm

Continue… Given T = 3 1 4 1 5 9 2 6 5 3 5 and P = 26

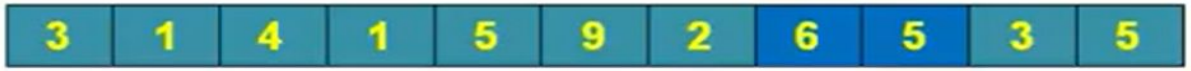
15 mod 11 = 4 , here 4 is equal to 4 → spurious hit

59 mod 11 = 4 equal to 4 → spurious hit

92 mod 11 = 4 equal to 4 → spurious hit



26 mod 11 = 4 equal to 4 → an exact match!!



65 mod 11 = 10 not equal to 4

Rabin Karp algorithm

Continue..

53 mod 11 = 9 , here 9 is not equal to 4, So don't Compare.

35 mod 11 = 2 not equal to 4

As we can see, when a match is found, further testing is done to insure that a match has indeed been found.

Rabin Karp algorithm

Example-2

- T = B A E C D E A A D A C
- P = A D A

- Here first give unique number to each character such as

A=1 , B=2 , C=3 , D=4 , E=5

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

1	2	3
A	D	A
1	4	1

- **First fine Hash values of Pattern :**
  - 141 mod 11 = 9

Rabin Karp algorithm

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

215 mod 11 = 6 , here 6 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

153 mod 11 = 10 , here 10 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

534 mod 11 = 6 , here 6 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

345 mod 11 = 4 , here 4 is not equal to 9 , So Don't Compare

Rabin Karp algorithm

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

451 mod 11 = 0 , here 0 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

511 mod 11 = 5 , here 5 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

114 mod 11 = 4 , here 4 is not equal to 9 , So Don't Compare

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

141 mod 11 = 9 , here 9 is equal to 9 , So Compare each Characters

Rabin Karp algorithm

1	2	3	4	5	6	7	8	9	10	11
B	A	E	C	D	E	A	A	D	A	C
2	1	5	3	4	5	1	1	4	1	3

413 mod 11 = 6 , here 6 is not equal to 9 , So Don't Compare

Pattern is Found at index = 8

Rabin Karp algorithm

Complexity:

The running time of RABIN-KARP-MATCHER in the worst case scenario O ((n-m+1) m

But it has a good average case running time

Rabin Karp algorithm

Complexity:

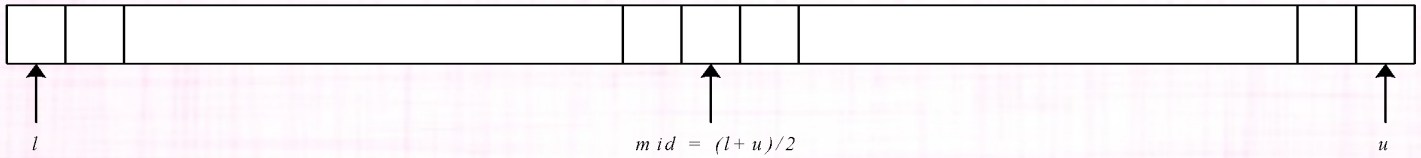
The running time of RABIN-KARP-MATCHER in the worst case scenario O ((n-m+1) m

But it has a good average case running time

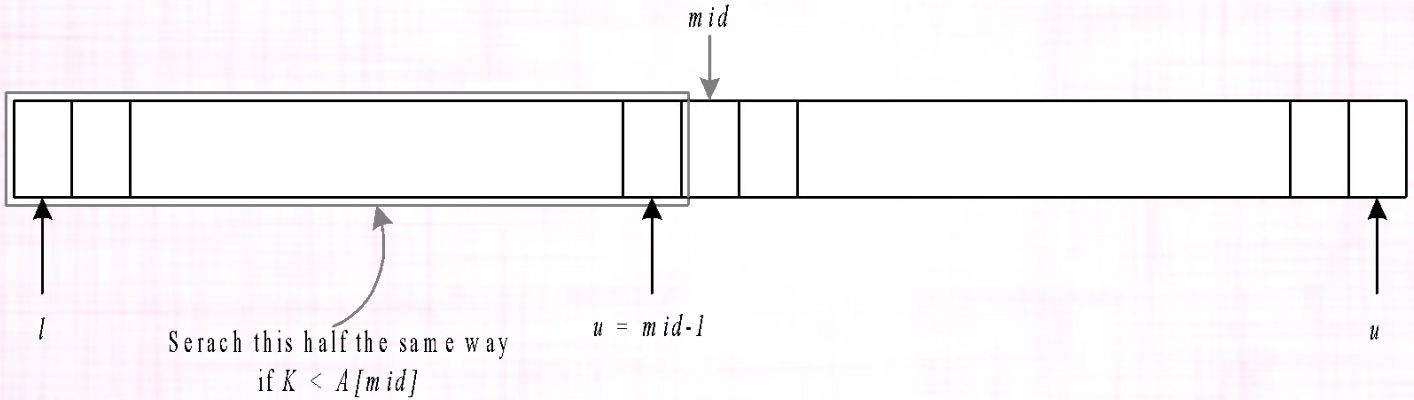
Binary Search Algorithm

is a searching algorithm used in a sorted array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).

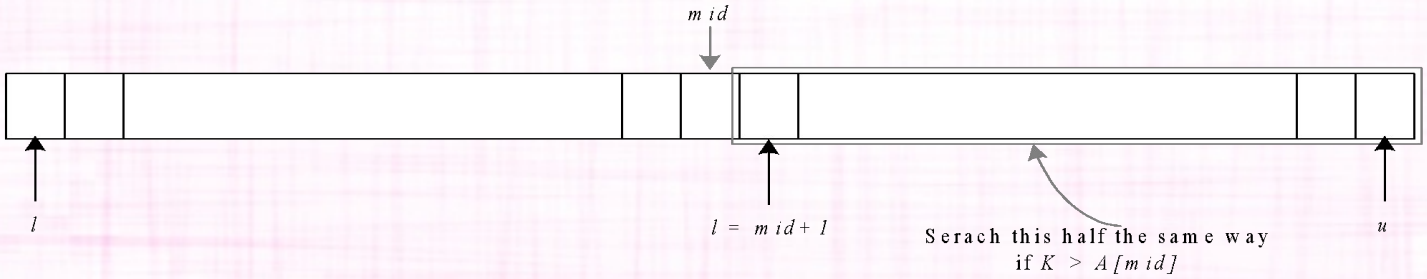
Technique



(a) An ordered array of elemnets with index values *l*, *u* and *mid*



(b) Search the entire list turns into the searching of left-half only

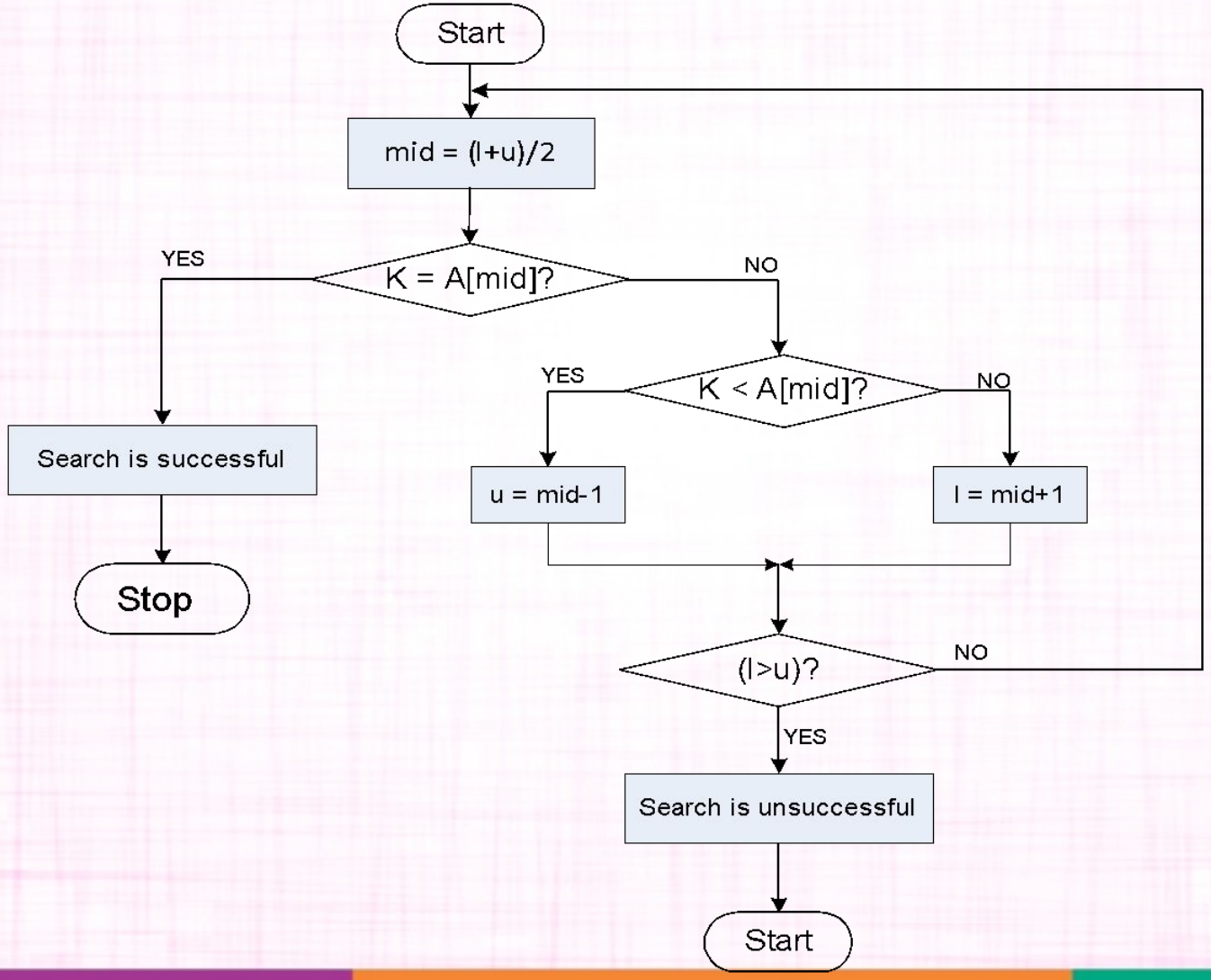


(c) Search the entire list turns into the searching of right-half only

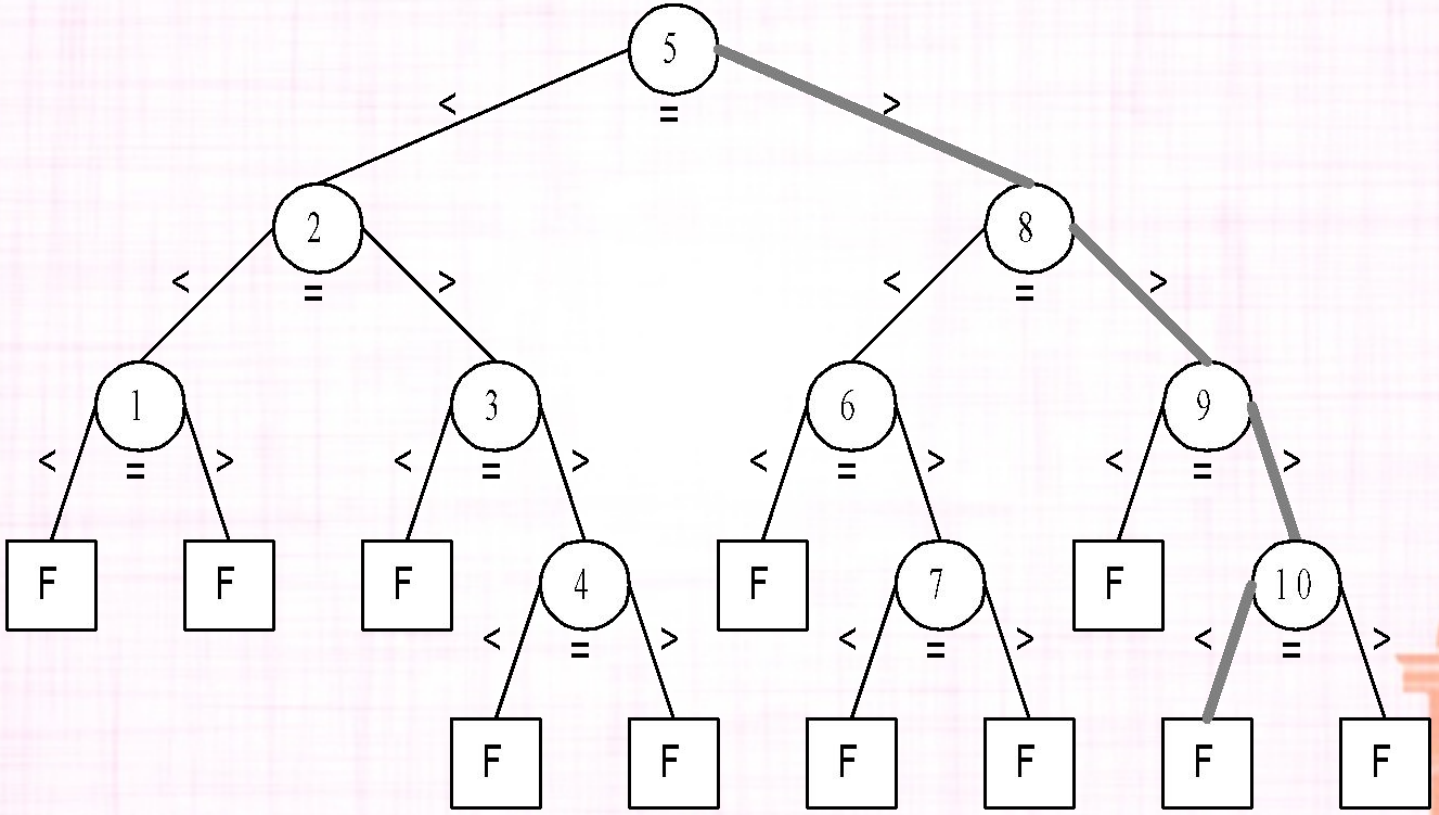
Algorithm

```
binarySearch(arr, key, low, high)
   if low > high
      return false
   else
      mid = low + (high - low) / 2
      if x == arr[mid]
         return mid


      /* x is on the right side */
      else if x > arr[mid]
         /* make a recursive call on the right half */
         return binarySearch(arr, x, mid + 1, high)


      /* x is on the left side */
      else
         /* make a recursive call on the left half  */
         return binarySearch(arr, x, low, mid - 1)
```

Flowchart


Start
mid = (l+u)/2
K = A[mid]?
YES
NO
Search is successful
Stop
K < A[mid]?
YES
NO
u = mid-1
l = mid+1
(l>u)?
NO
YES
Search is unsuccessful
Start

Complexity Analysis



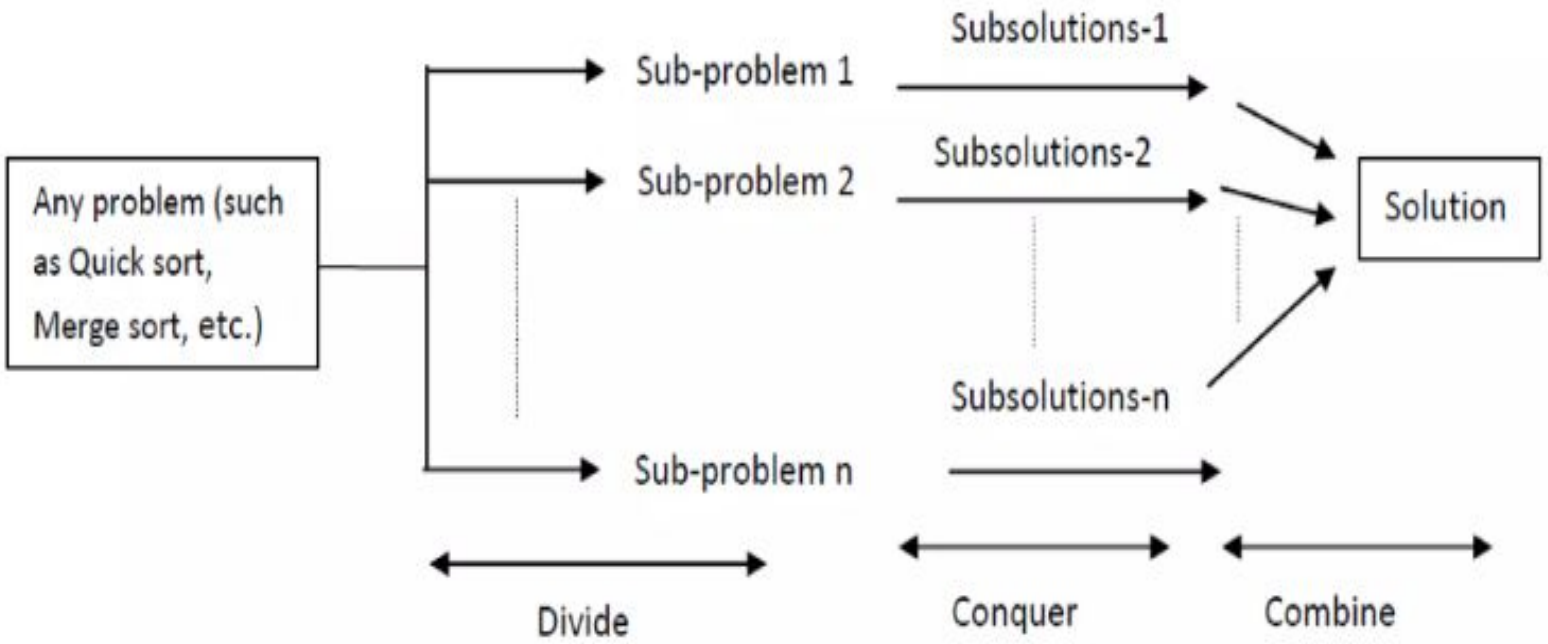
Let n be the total number of elements in the list under search and there exist an integer k such that:-

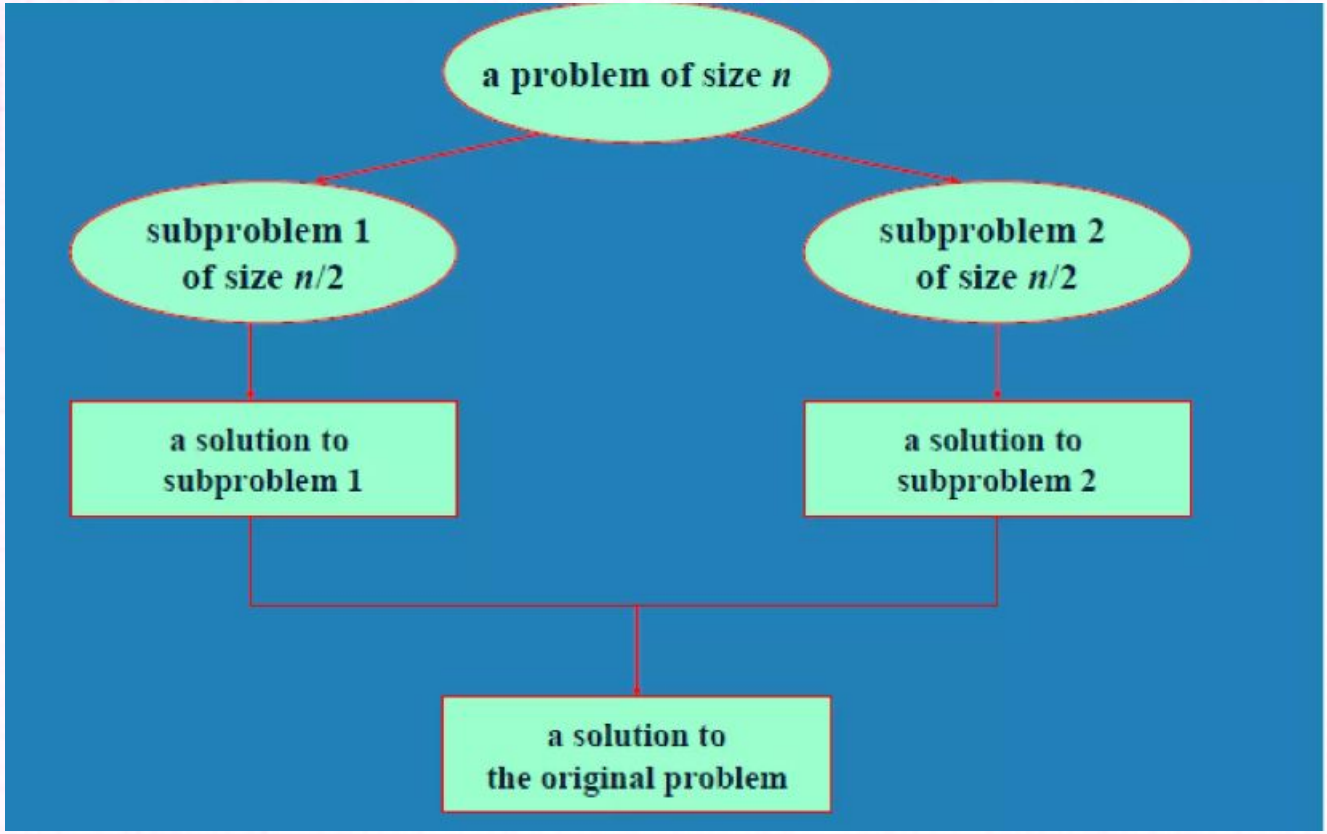
- For successful search:-
- If $2^{k-1} \le n < 2^k$, then the binary search algorithm requires at least one comparison and at most k comparisons.
- For unsuccessful search:-
- If $n = 2^{k-1}$, then the binary search algorithm requires k comparisons.
- If $2^{k-1} \le n < 2^k - 1$, then the binary search algorithm requires either k-1 or k number of comparisons

DIVIDE AND CONQUER

- **Divide and Conquer Algorithm** involves breaking a larger problem into smaller subproblems, solving them independently, and then combining their solutions to solve the original problem.

- The basic idea is to recursively divide the problem into smaller subproblems until they become simple enough to be solved directly. Once the solutions to the subproblems are obtained, they are then combined to produce the overall solution.


Any problem (such as Quick sort, Merge sort, etc.)
Sub-problem 1
Sub-problem 2
Sub-problem n
Subsolutions-1
Subsolutions-2
Subsolutions-n
Solution
Divide
Conquer
Combine


a problem of size n
subproblem 1 of size n/2
subproblem 2 of size n/2
a solution to subproblem 1
a solution to subproblem 2
a solution to the original problem

Divide & Conquer Algo

1. Algo Dand C(P)
2. {
3. If small (P) then return S(P)
4. else
5. {
6. Divide P into smaller instances $P_1, P_2 \ldots P_k$, k>=1
7. Apply Dand C to each of these problem
8. Return combine (Dand C(P_1), Dand C(P_2)…Dand C(P_k);
9. }
10. }

The complexity of divide and conquer algo is given by recurrence equation

$$T(n) = \begin{cases} T(1) & n=1 \\ aT(n/b) + f(n) & n>1 \end{cases}$$

- Merge Sort Algorithm:
  - **Divide**: If S has at leas two elements (nothing needs to be done if S has zero or one elements), remove all the elements from S and put them into two sequences, S_1 and S_2, each containing about half of the elements of S. (i.e. S_1 contains the first $\lceil n/2 \rceil$ elements and S_2 contains the remaining $\lfloor n/2 \rfloor$ elements.
  - **Recur**: Recursive sort sequences S_1 and S_2.
  - **Conquer**: Put back the elements into S by merging the sorted sequences S_1 and S_2 into a unique sorted sequence.
- Merge Sort Tree:
  - Take a binary tree T
  - Each node of T represents a recursive call of the merge sort algorithm.
  - We associate with each node v of T a the set of input passed to the invocation v represents.
  - The external nodes are associated with individual elements of S, upon which no recursion is called.

mergeSort(arr[], l, r)

If l < r

1. Find the middle point to divide the array into two halves: middle m = (l+r)/2

2. Call mergeSort for first half: Call mergeSort(arr, l, m)

3. Call mergeSort for second half: Call mergeSort(arr, m+1, r)

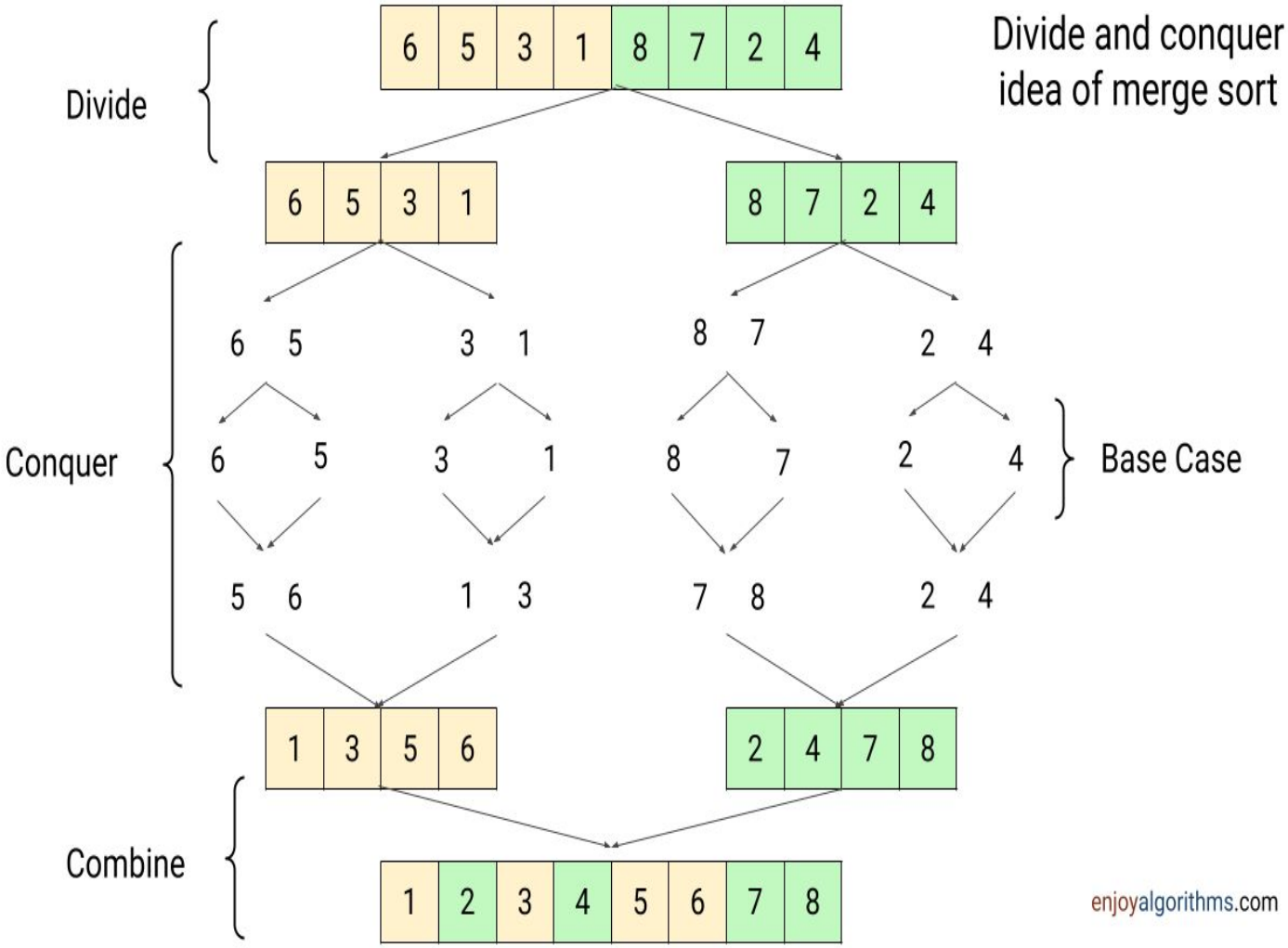
4. Merge the two halves sorted in step 2 and 3: Call merge(arr, l, m, r)

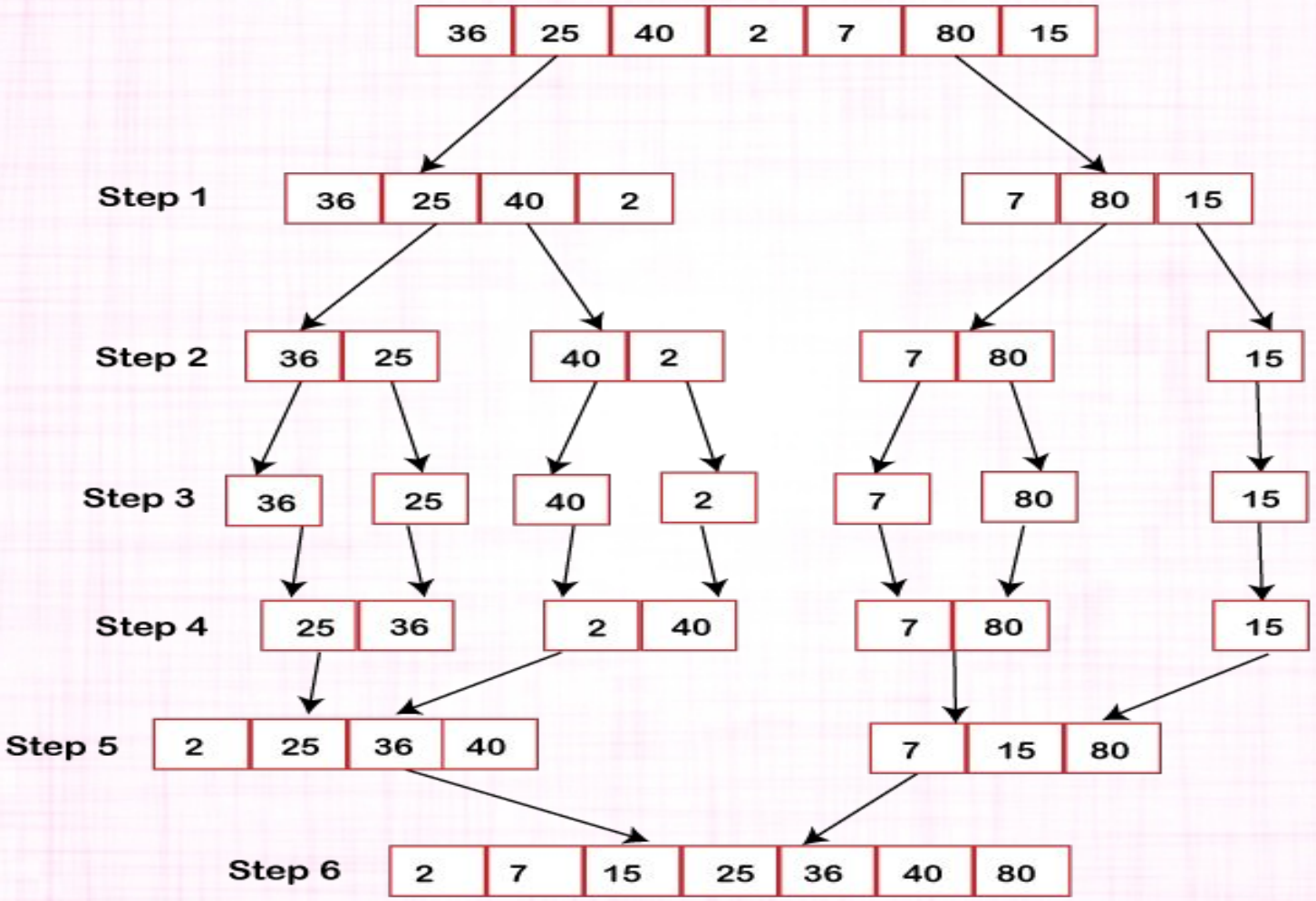
Algorithm Merge-Sort

```
ALGORITHM-MERGE SORT

1. If p<r

2. Then q ← ( p+ r)/2

3. MERGE-SORT (A, p, q)

4. MERGE-SORT ( A, q+1,r)

5. MERGE ( A, p, q ,r)
```

FUNCTIONS: MERGE (A, p, q, r)

```
1. n 1 = q-p+1
2. n 2= r-q
3. create arrays [1.....n 1 + 1] and R [ 1.....n 2 +1 ]
4. for i ← 1 to n 1
5. do [i] ← A [ p+ i-1]
6. for j ← 1 to n2
7. do R[j] ← A[ q + j]
8. L [n 1+ 1] ← ∞
9. R[n 2+ 1] ← ∞
10. I ← 1
11. J ← 1
12. For k ← p to r
13. Do if L [i] ≤ R[j]
14. then A[k] ← L[ i]
15. i ← i +1
16. else A[k] ← R[j]
17. j ← j+1
```


Divide and conquer idea of merge sort
Divide
6 5 3 1 8 7 2 4
6 5 3 1
8 7 2 4
Conquer
6 5
3 1
8 7
2 4
6
5
3
1
8
7
2
4
Base Case
5 6
1 3
7 8
2 4
1 3 5 6
2 4 7 8
Combine
1 2 3 4 5 6 7 8
enjoyalgorithms.com


36 25 40 2 7 80 15
Step 1
36 25 40 2
7 80 15
Step 2
36 25
40 2
7 80
15
Step 3
36
25
40
2
7
80
15
Step 4
25 36
2 40
7 80
15
Step 5
2 25 36 40
7 15 80
Step 6
2 7 15 25 36 40 80

Analysis of Merge Sort

Running time $T(n)$ of Merge Sort:

Divide: computing the middle takes $\Theta(1)$

Conquer: solving 2 sub-problems takes $2T(n/2)$

Combine: merging n elements takes $\Theta(n)$

Total:

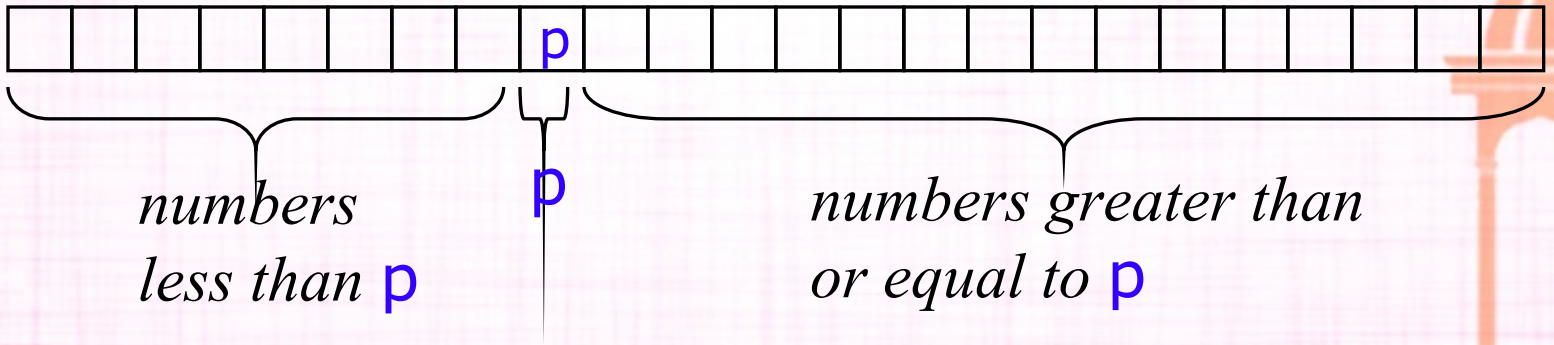
$$T(n) = \Theta(1) \quad \text{if } n = 1$$
$$T(n) = 2T(n/2) + \Theta(n) \quad \text{if } n > 1$$

$$\begin{aligned} T(n) &= 2\,T(n/2) + n \\ &= 2\,((n/2)\log(n/2) + (n/2)) + n \\ &= n\,(\log(n/2)) + 2n \\ &= n \log n - n + 2n \\ &= n \log n + n \\ &= O(n \log n) \end{aligned}$$

	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Quicksort I: Basic idea

- Pick some number p from the array
- Move all numbers less than p to the beginning of the array
- Move all numbers greater than (or equal to) p to the end of the array
- Quicksort the numbers less than p
- Quicksort the numbers greater than or equal to p



Quicksort I

- To sort a[left...right]:

1. if left < right:

1.1. Partition a[left...right] such that:

all a[left...p-1] are less than a[p], and

all a[p+1...right] are >= a[p]

1.2. Quicksort a[left...p-1]

1.3. Quicksort a[p+1...right]

2. Terminate

Partitioning II

- Choose an array value (say, the first) to use as the pivot
- Starting from the left end, find the first element that is greater than or equal to the pivot
- Searching backward from the right end, find the first element that is less than the pivot
- Interchange (swap) these two elements
- Repeat, searching from where we left off, until done

Partitioning

- To partition a[left...right]:

1. Set pivot = a[left], l = left + 1, r = right;
2. while l < r, do
2.1. while l < right & a[l] < pivot , set l = l + 1
2.2. while r > left & a[r] >= pivot , set r = r - 1
2.3. if l < r, swap a[l] and a[r]
3. Set a[left] = a[r], a[r] = pivot
4. Terminate

Example of partitioning

- choose pivot: 4 3 6 9 2 4 3 1 2 1 8 9 3 5 6
- search: 4 3 6 9 2 4 3 1 2 1 8 9 3 5 6
- swap: 4 3 3 9 2 4 3 1 2 1 8 9 6 5 6
- search: 4 3 3 9 2 4 3 1 2 1 8 9 6 5 6
- swap: 4 3 3 1 2 4 3 1 2 9 8 9 6 5 6
- search: 4 3 3 1 2 4 3 1 2 9 8 9 6 5 6
- swap: 4 3 3 1 2 2 3 1 4 9 8 9 6 5 6
- search: 4 3 3 1 2 2 3 1 4 9 8 9 6 5 6 (left > right)
- swap with pivot: 1 3 3 1 2 2 3 4 4 9 8 9 6 5 6

```
QUICKSORT(A, p, r)
1  if p < r
2      then q ← PARTITION(A, p, r)
3           QUICKSORT(A, p, q − 1)
4           QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x ← A[r]
2  i ← p − 1
3  for j ← p to r − 1
4        do if A[j] ≤ x
5              then i ← i + 1
6                   exchange A[i] ↔ A[j]
7  exchange A[i + 1] ↔ A[r]
8  return i + 1
```

Typical case for quicksort

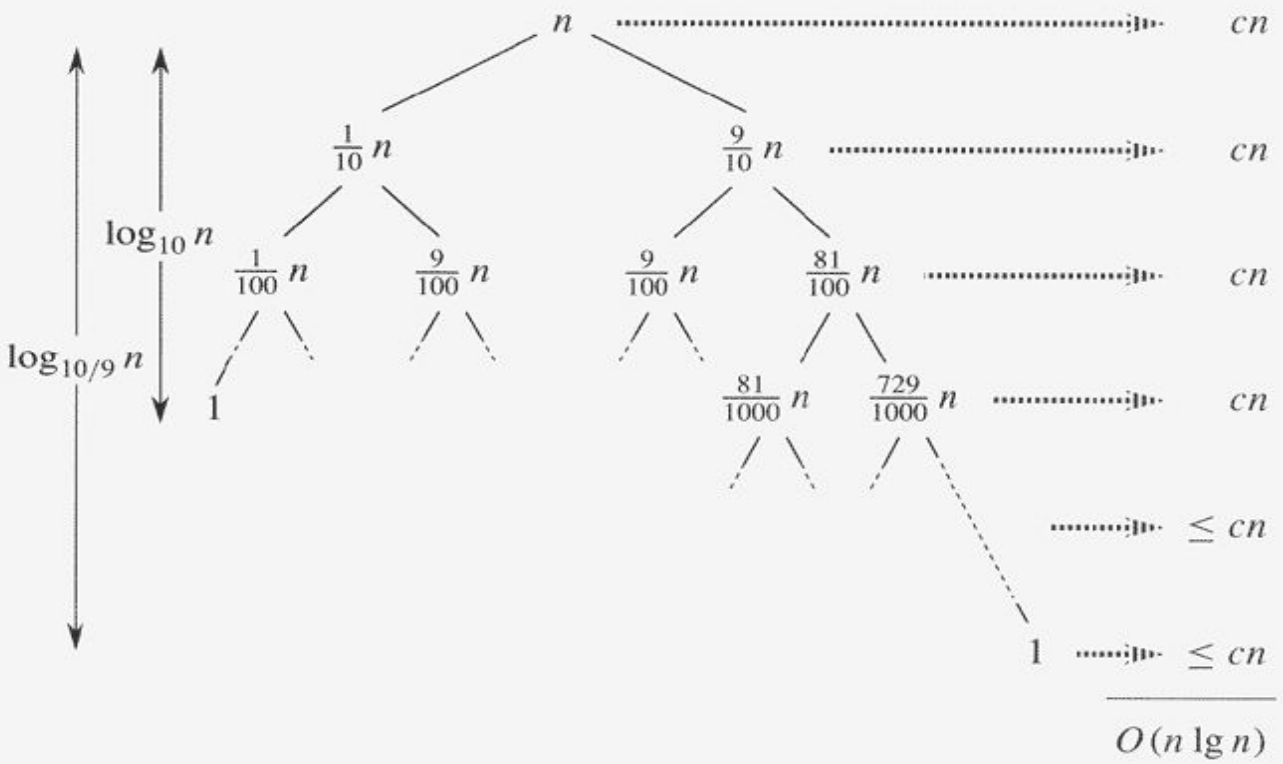


Figure 7.4 A recursion tree for QUICKSORT in which PARTITION always produces a 9-to-1 split, yielding a running time of $O(n \lg n)$. Nodes show subproblem sizes, with per-level costs on the right. The per-level costs include the constant c implicit in the $\Theta(n)$ term.

Typical case for quicksort

- If the array is sorted to begin with, Quicksort is terrible: $O(n^2)$
- It is possible to construct other bad cases
- However, Quicksort is *usually* $O(n \log_2 n)$
- The constants are so good that Quicksort is generally the fastest algorithm known
- Most real-world sorting is done by Quicksort

Strassen's Algorithm

Strassen's Matrix Multiplication is the divide and conquer approach to solve the matrix multiplication problems. The usual matrix multiplication method multiplies each row with each column to achieve the product matrix. The time complexity taken by this approach is $\mathbf{O(n^3)}$, since it takes two loops to multiply. Strassen's method was introduced to reduce the time complexity from $\mathbf{O(n^3)}$ to $\mathbf{O(n^{\log 7})}$.

Basic Matrix Multiplication

Suppose we want to multiply two matrices of size N x N: for example A x $B = C$.

$$\begin{vmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{vmatrix} = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix} \begin{vmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{vmatrix}$$

$C_{11} = a_{11}b_{11} + a_{12}b_{21}$
$C_{12} = a_{11}b_{12} + a_{12}b_{22}$
$C_{21} = a_{21}b_{11} + a_{22}b_{21}$
$C_{22} = a_{21}b_{12} + a_{22}b_{22}$

2x2 matrix multiplication can be accomplished in 8 multiplication.($2^{\log_2 8} = 2^3$)

Divide and Conquer Matrix Multiply

$$A \times B = R$$

$$\begin{bmatrix} A_0 & A_1 \\ A_2 & A_3 \end{bmatrix} \times \begin{bmatrix} B_0 & B_1 \\ B_2 & B_3 \end{bmatrix} = \begin{bmatrix} A_0 \times B_0 + A_1 \times B_2 & A_0 \times B_1 + A_1 \times B_3 \\ A_2 \times B_0 + A_3 \times B_2 & A_2 \times B_1 + A_3 \times B_3 \end{bmatrix}$$

- Divide matrices into sub-matrices: A_0 , A_1, A_2 etc
- Use blocked matrix multiply equations
- Recursively multiply sub-matrices

Matrix multiplication (Divide-conquer)

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} A_{00} * B_{00} + A_{01} * B_{10} & A_{00} * B_{01} + A_{01} * B_{11} \\ A_{10} * B_{00} + A_{11} * B_{10} & A_{10} * B_{01} + A_{11} * B_{11} \end{bmatrix}$$

A, B: n by n matrices;

A_{ij}, B_{ij}: four square submatrix having dimensions n/2 by n/2 matrices, where i, j ∈ {0, 1}

$$T(n) = \begin{cases} b & n =< 2 \\ 8T(n/2) + cn^2 & n > 2 \end{cases}$$

multiplication: $\Theta(n^3)$ addition: $\Theta(n^3)$

Strassens's Matrix Multiplication

$$\begin{vmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{vmatrix} = \begin{vmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{vmatrix} \begin{vmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{vmatrix}$$

$P_1 = (A_{11} + A_{22})(B_{11} + B_{22})$
$P_2 = (A_{21} + A_{22}) * B_{11}$
$P_3 = A_{11} * (B_{12} - B_{22})$
$P_4 = A_{22} * (B_{21} - B_{11})$
$P_5 = (A_{11} + A_{12}) * B_{22}$
$P_6 = (A_{21} - A_{11}) * (B_{11} + B_{12})$
$P_7 = (A_{12} - A_{22}) * (B_{21} + B_{22})$

$C_{11} = P_1 + P_4 - P_5 + P_7$
$C_{12} = P_3 + P_5$
$C_{21} = P_2 + P_4$
$C_{22} = P_1 + P_3 - P_2 + P_6$

Strassen's Matrix Multiplication

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

$M_1=(A_{00}+A_{11})*(B_{00}+B_{11})$
$M_2=(A_{10}+A_{11})*B_{00}$
$M_3=A_{00}*(B_{01}-B_{11})$
$M_4=A_{11}*(B_{10}-B_{00})$
$M_5=(A_{00}+A_{01})*B_{11}$
$M_6=(A_{10}-A_{00})*(B_{00}+B_{01})$
$M_7=(A_{01}-A_{11})*(B_{10}+B_{11})$

recurrence relations:

multiplication: M(n) = 7

addition: A(n) = 18

Strassen's Matrix Multiplication

$$\begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix} = \begin{bmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{bmatrix} * \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \end{bmatrix}$$

$$= \begin{bmatrix} M_1 + M_4 - M_5 + M_7 & M_3 + M_5 \\ M_2 + M_4 & M_1 + M_3 - M_2 + M_6 \end{bmatrix}$$

$M_1=(A_{00}+A_{11})*(B_{00}+B_{11})$
$M_2=(A_{10}+A_{11})*B_{00}$
$M_3=A_{00}*(B_{01}-B_{11})$
$M_4=A_{11}*(B_{10}-B_{00})$
$M_5=(A_{00}+A_{01})*B_{11}$
$M_6=(A_{10}-A_{00})*(B_{00}+B_{01})$
$M_7=(A_{01}-A_{11})*(B_{10}+B_{11})$

$$T(n) = \begin{cases} b & n =< 2 \\ 7T(n/2) + an^2 & n > 2 \end{cases}$$

$M(n) \approx \Theta(n^{2.807})$

$A(n) \approx \Theta(n^{2.807})$

Time Analysis

$$T(1) = 1 \qquad (\text{assume } N = 2^k)$$

$$T(N) = 7T(N/2)$$

$$T(N) = 7^k T(N/2^k) = 7^k$$

$$T(N) = 7^{\log N} = N^{\log 7} = N^{2.81}$$

Thank you